Recitation 1: Optimizing Matrix Multiply

Julian Bellavita, 1/29/26

Matrix Multiply Basics

- Dense matrix multiply aka GEMM (General Matrix Multiply)

$$C = C + AB$$

- Three input matrices — A, B, C — all are square with dimension n
- This kernel powers lots of applications
  - Dense linear algebra, sparse linear algebra, machine learning, others…

Matrix Multiply Basics

- Each entry of C is a dot product of a row of A and a column of B

$$C(i,j) = \sum_{k=1}^{n} A(i,k)B(k,j)$$

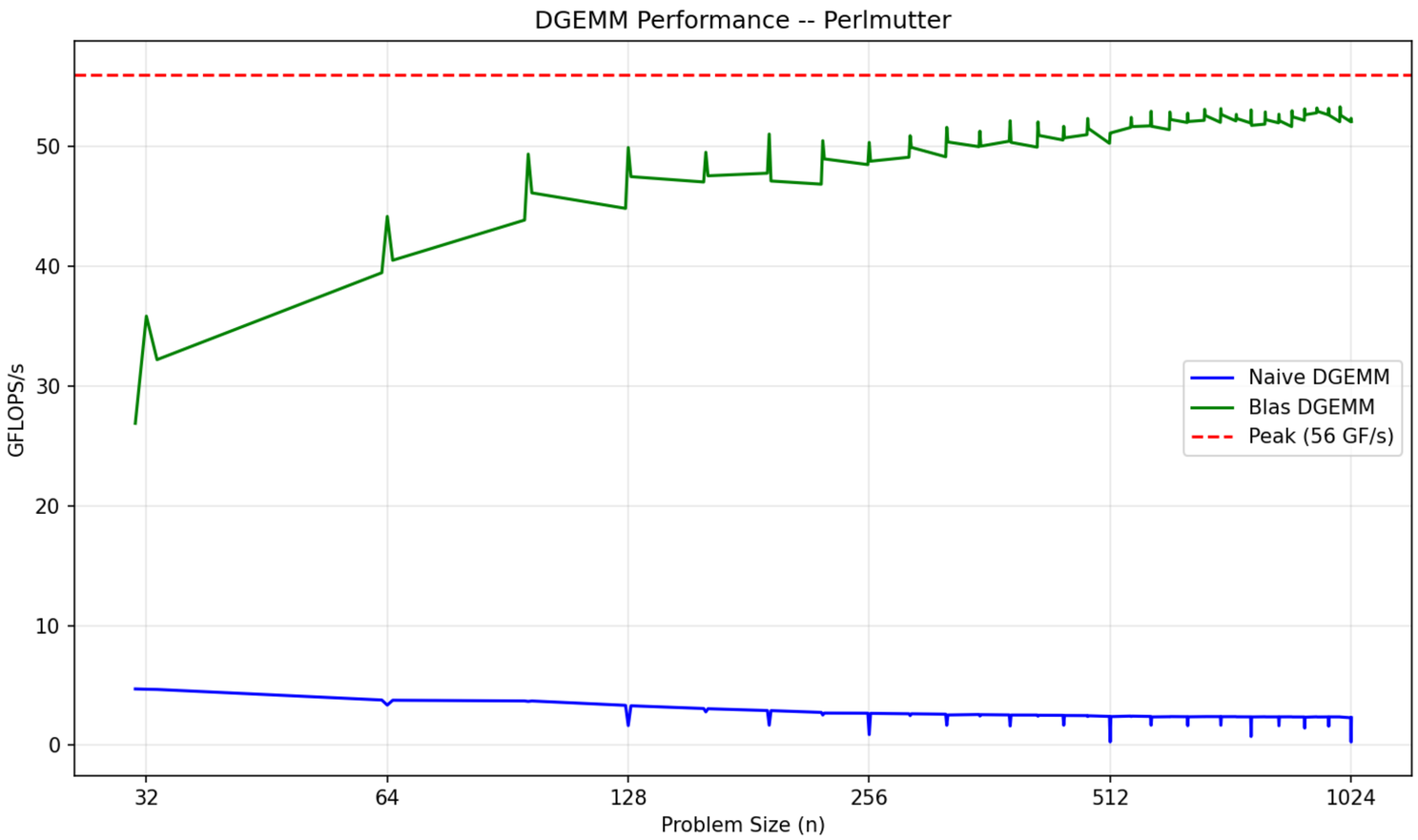
- All n^2 pairs of rows of A and columns of B — take their dot product
- O(n^3) FLOPS

Naive Matrix Multiply

- Simplest possible implementation of matrix multiply in C

```
void gemm(int n, double *A, double *B, double *C) {
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            for (int k=0; k<n; k++) {
                C[i + j * n] += A[i + k * n] * B[k + j * n]  ;
            }
        }
    }
}
```

Naive Matrix Multiply is Slow



- The naive approach gets like 4% of peak — this is bad
- BLAS gets close to peak — so we can make GEMM fast

Assignment Details

- Your job: write a fast DGEMM kernel for a single CPU core (the D stands for “double precision”)
- Grading breakdown
  - 10/100 points — checkpoint
  - 70/100 points — performance
  - 20/100 points — report — a single performance plot and like 1k characters describing it — should be fast
- Performance is determined based on percentage of peak (56 GFLOPS/s on Perlmutter)
- Checkpoint: **by 2/5 you must get at least 10% of peak**

- >= 50.0% of peak performance: A+
- >= 40.0% of peak performance: A
- >= 35.0% of peak performance: A-
- >= 25.0% of peak performance: B+
- >= 15.0% of peak performance: B
- >= 10.0% of peak performance: B-
- < 10.0% of peak performance: C+

Perlmutter's Processors

- AMD EPYC 7763 — 64 cores per CPU (we only use 1)
- Cache sizes
  - L1: 64 KB
  - L2: 512 KB
  - L3: 256 MB (shared among all cores)
- AVX2 instruction set supported — can operate on 4 double precision values
- 16 256-bit vector registers

Optimizations

Optimization 1: Loop Ordering

- Underlying dense matrix storage is just a buffer
- We need a rule for mapping regions of the buffer to regions of the matrix
- **Column-major ordering**: traversing the buffer traverses columns of the matrix
- All matrices in this assignment are stored in **column-major order**

Row-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Column-major order

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Optimization 1: Loop Ordering

- Innermost loop traverses through a row of A — but A is column-major order => each one of these accesses is non-contiguous

- Bad for lots of reasons

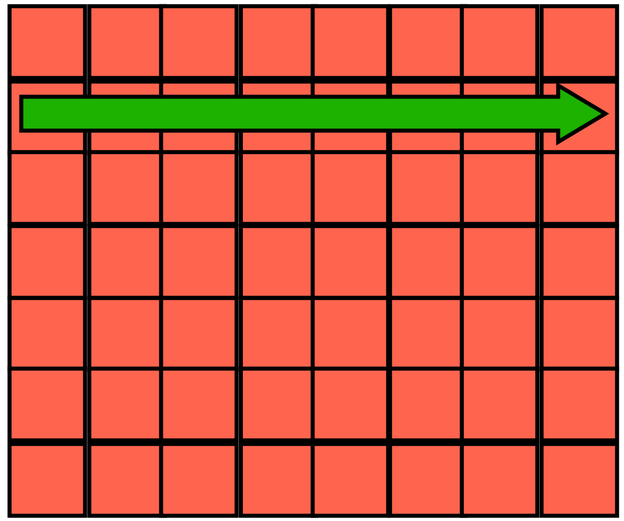
1. Each access incurs a cache miss

2. Each entry of the row may lie on a different page — you need n translation lookaside buffer (TLB) entries per row of A potentially

3. Harder to vectorize — elements of A accesses by subsequent iterations are non-contiguous

```
void gemm(int n, double *A, double *B, double *C) {
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            for (int k=0; k<n; k++) {
                C[i + j * n] += A[i + k * n] * B[k + j * n]  ;
            }
        }
    }
}
```

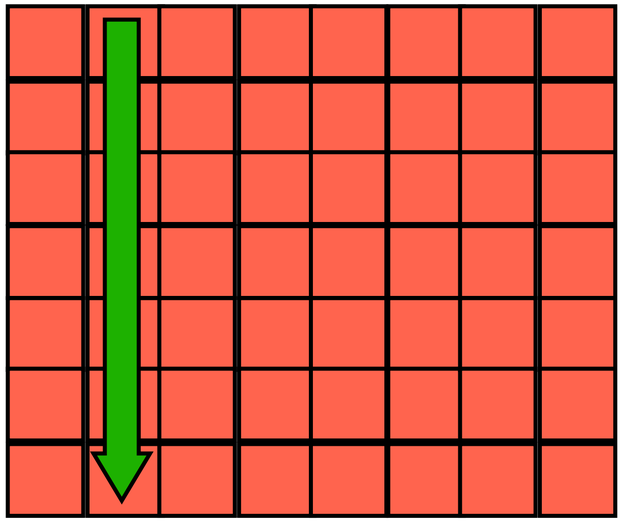
A



Optimization 1: Loop Ordering

- You can re-order the three loops without impacting correctness
- Reordering the i and k loops ensures elements of A are accessed contiguously
- Some loop orderings are better than others — experiment with many!

A



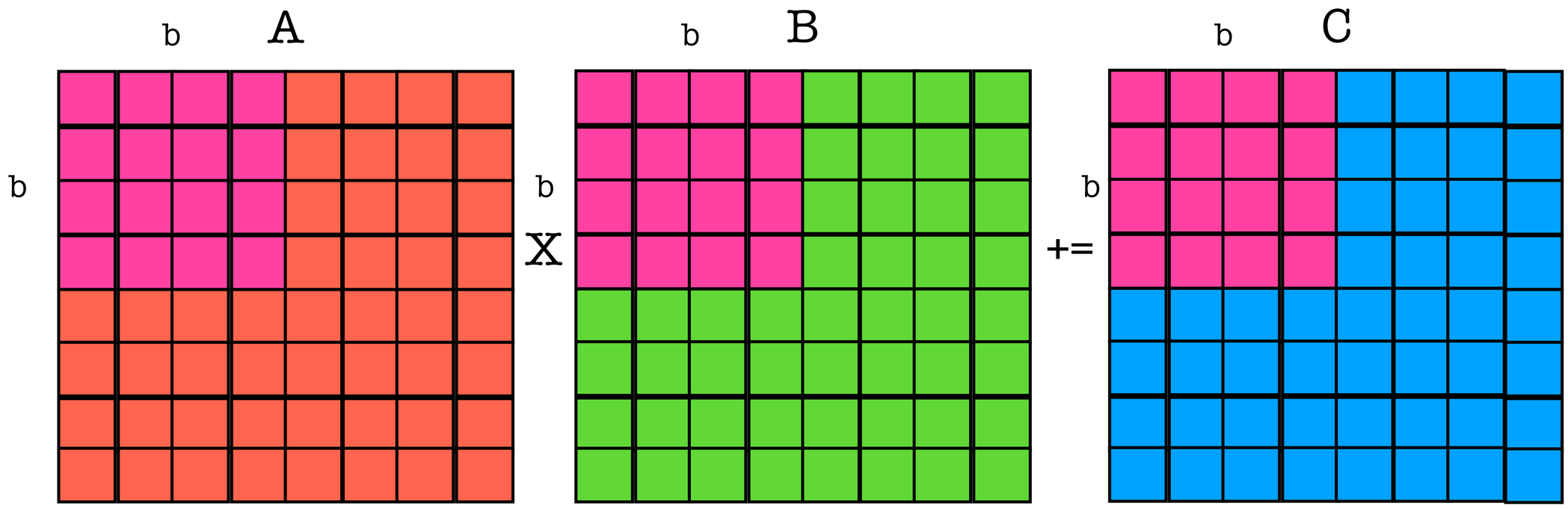
```
void gemm(int n, double *A, double *B, double *C) {
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            for (int k=0; k<n; k++) {
                C[i + j * n] += A[i + k * n] * B[k + j * n]  ;
            }
        }
    }
}
```

Reorder the i and k loops

```
void gemm_reorder(int n, double *A, double *B, double *C) {
    for (int k=0; k<n; k++) {
        for (int j=0; j<n; j++) {
            for (int i=0; i<n; i++) {
                C[i + j * n] += A[i + k * n] * B[k + j * n]  ;
            }
        }
    }
}
```

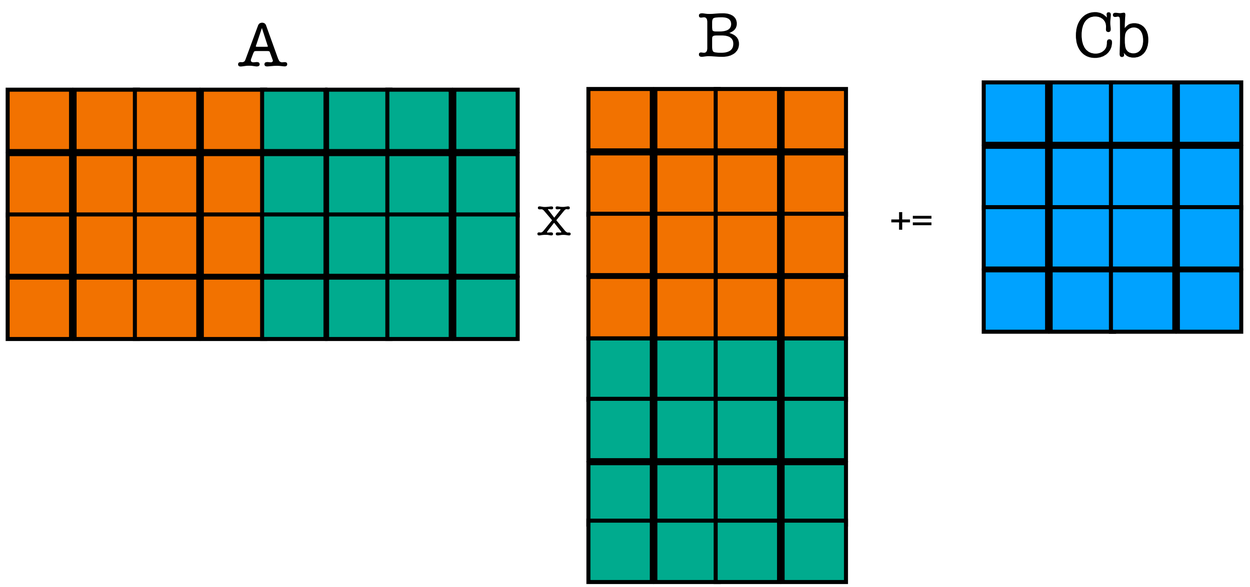
Optimization 2: Cache Blocking

- Core idea: split the matrix into `bxb` blocks that can fit in L1 or L2 cache — perform all computation with those blocks while they're in the cache
- This is good because subsequent accesses to that block will involve the cache instead of main memory
- **This is beneficial for GEMM because there's lots of data re-use — just need to make sure the re-used data is small enough to fit in cache**

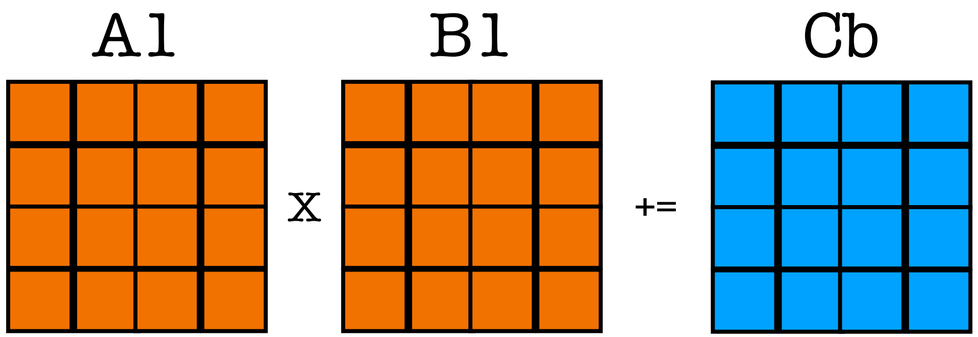


Optimization 2: Cache Blocking

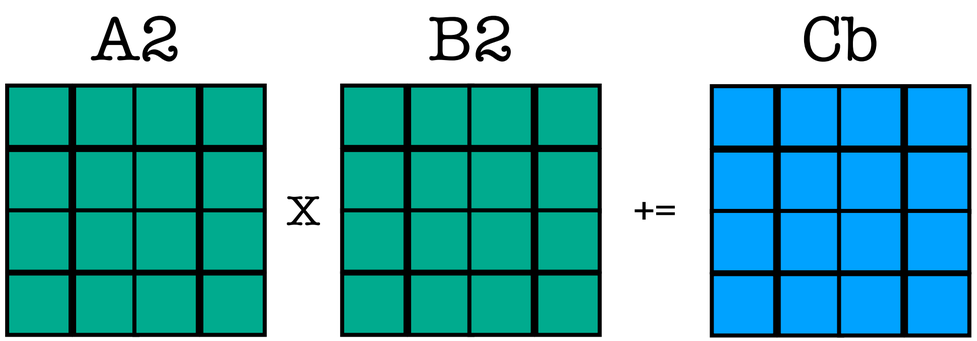
- Example — blocked inner product



Iteration 1



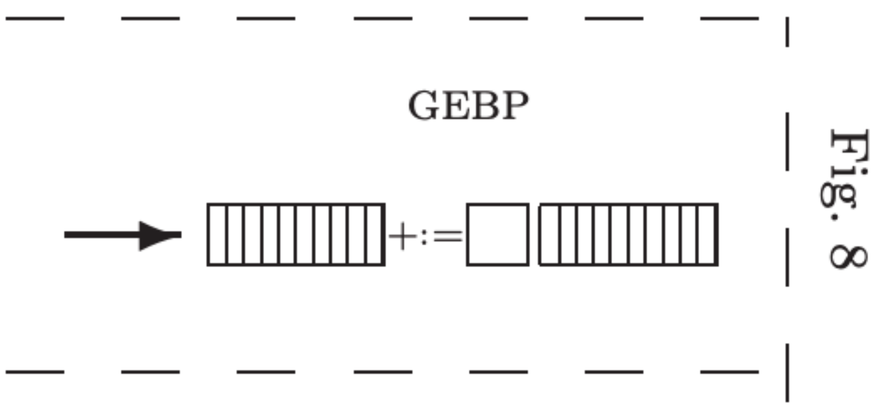
Iteration 2



During iteration 2, Cb is already in the cache

Optimization 2: Cache Blocking

- The starter code already does this “inner-product” style of blocking for you — the reuse here is in blocks of C
- There are other blocking strategies that enable re-use in the other matrices — see GotoBLAS paper
- You need to pick the block size — ideally, a block of A, B, C should all fit in L2 or L1 cache
- Try different block sizes — see “Analytical Modeling is enough for High-Performance BLIS” paper
- Experiment with different loop orderings within the block multiplications

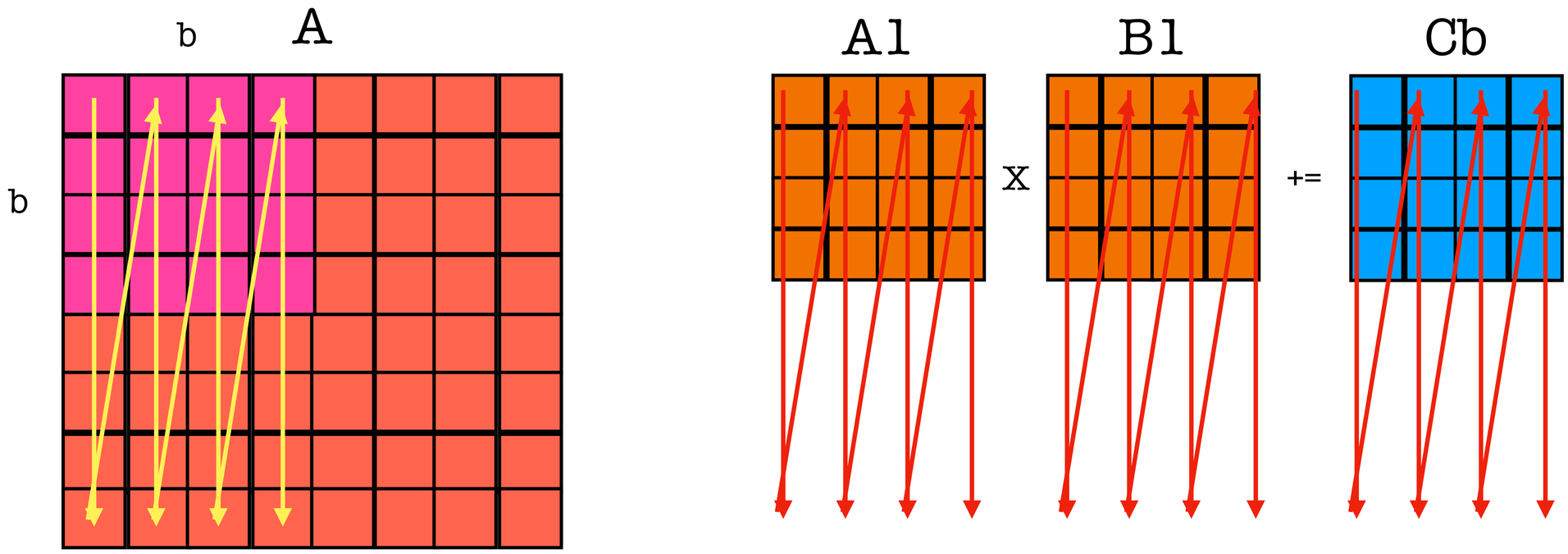


Blocking strategy that exploits reuse in A

Source: https://www.cs.utexas.edu/~flame/pubs/GotoTOMS_revision.pdf

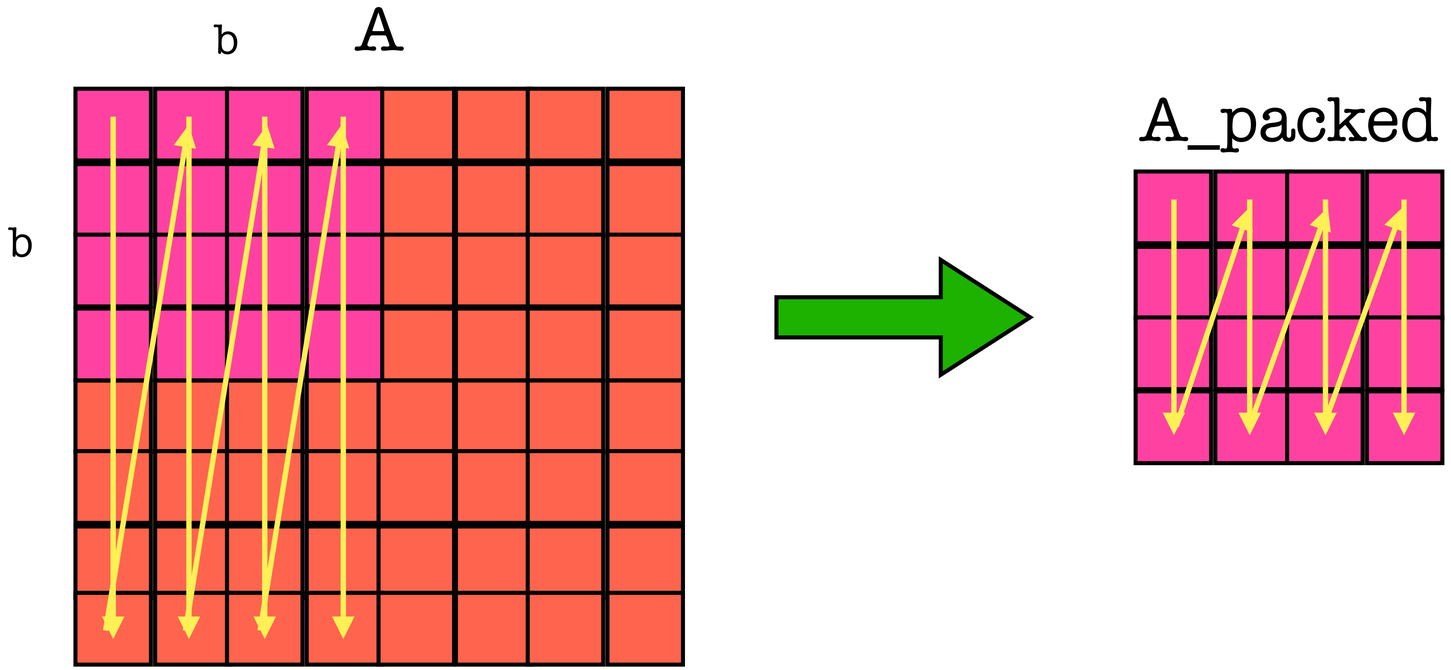
Optimization 3: Packing

- The whole matrix is column-major order, but individual blocks are not in column major order
- The block multiplications from earlier will involve non-contiguous accesses with stride n - b



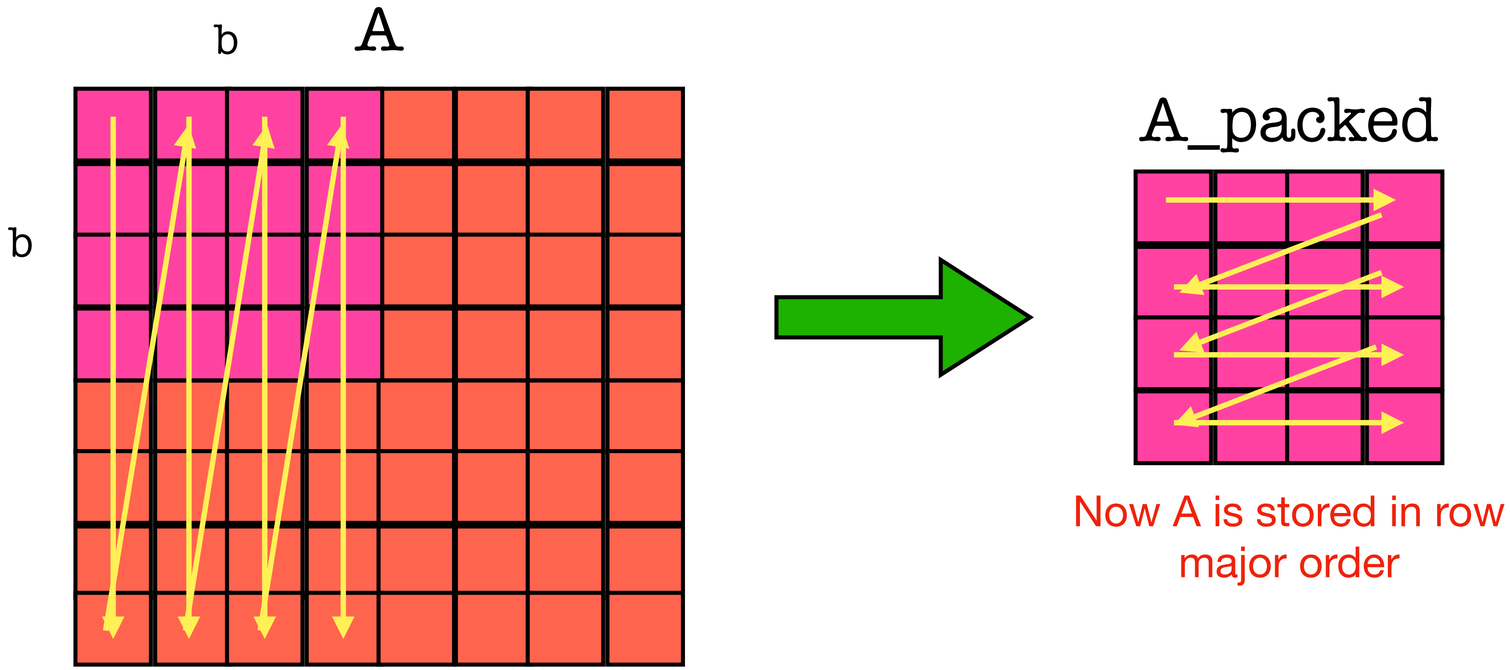
Optimization 3: Packing

- **Packing**: copy blocks of the matrices into a separate buffer where they are stored contiguously
- Subsequent tile multiplications involve contiguous accesses



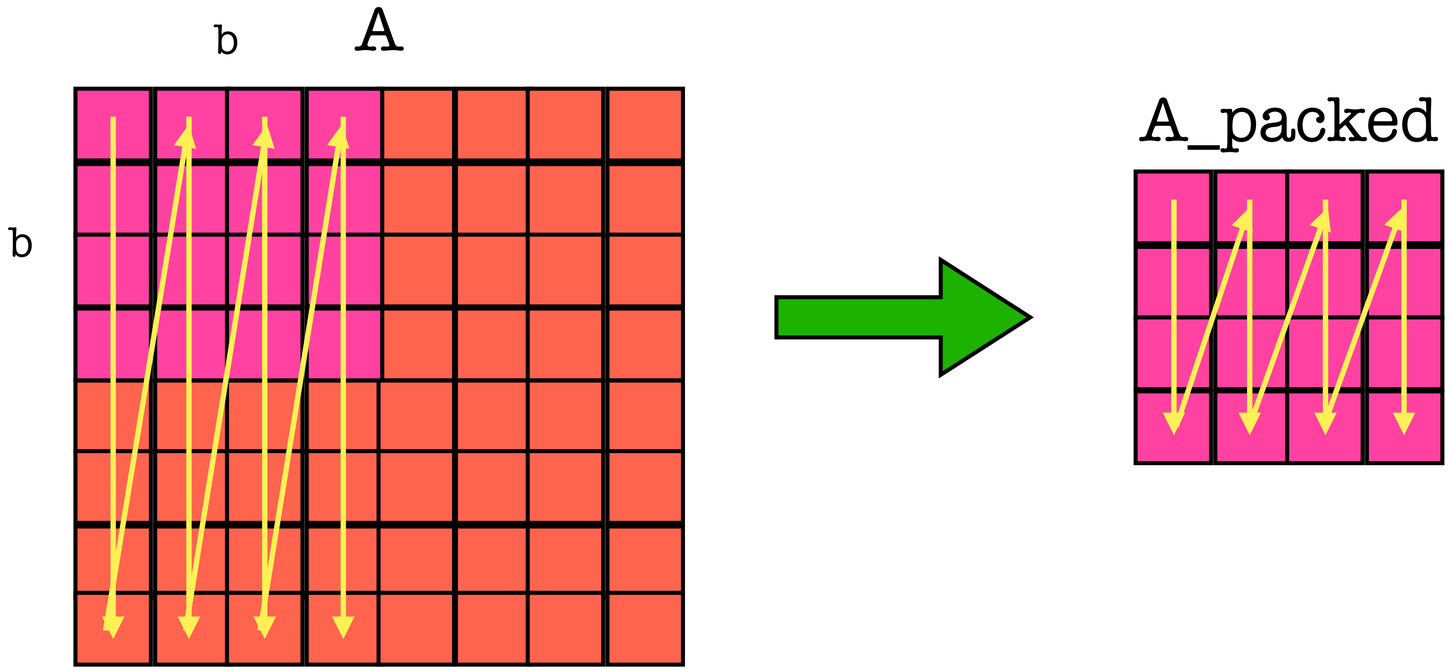
Optimization 3: Packing

- You can also pack the block so it is stored in any way you want — it doesn't have to be column major order
- This can interact with the microkernel (optimization 4) in some interesting ways



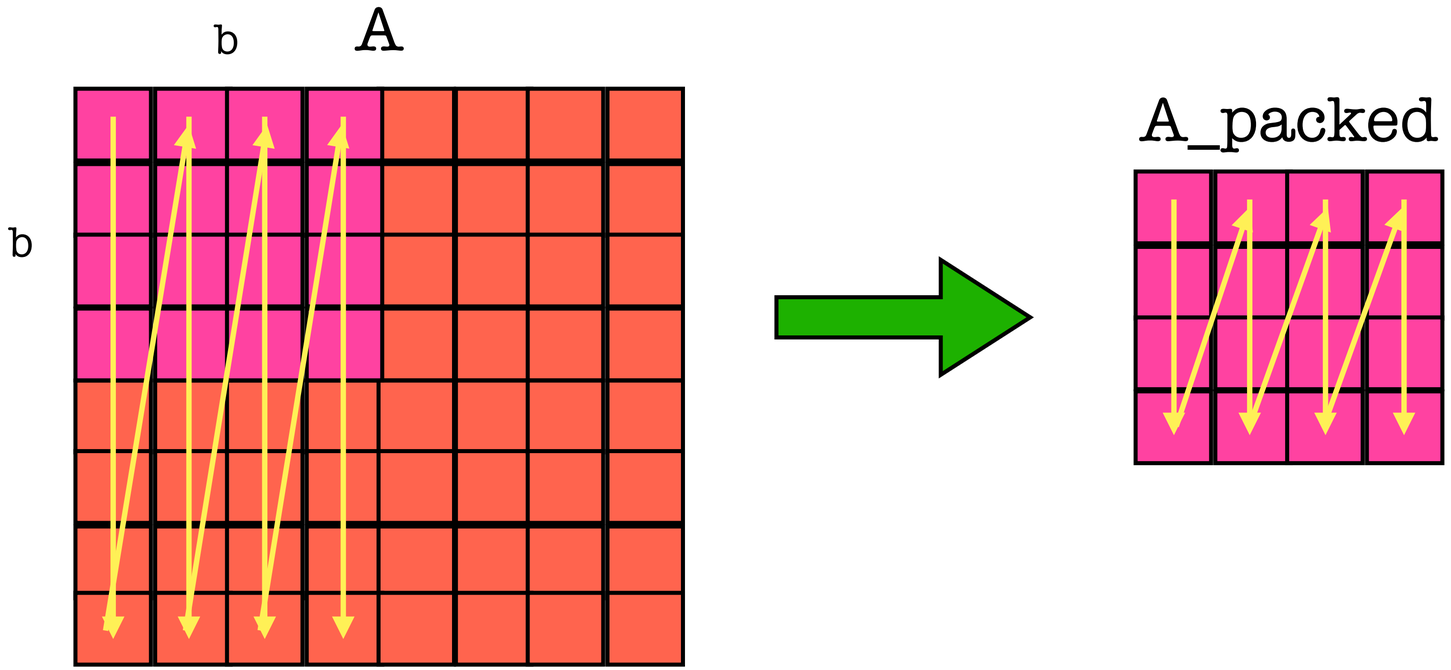
Optimization 3: Packing

- It's only worth it to pack a matrix block if it's re-used/accessed a lot — this is needed to justify the overhead of packing
- For our block-inner product approach, packing C can be quite beneficial



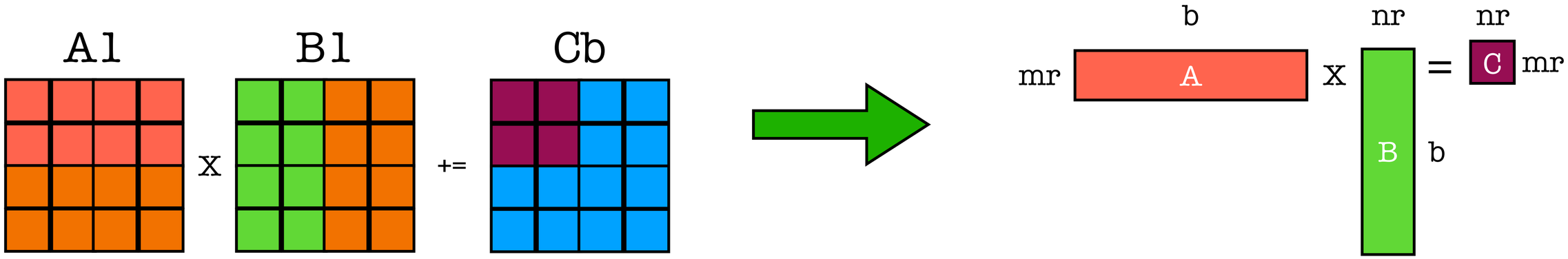
Optimization 3: Packing

- Advice: use `_mm_malloc` from `<immintrin.h>` when allocating packed buffers so that your packed matrix buffers are aligned to 32 bytes
- This makes vector load/stores faster (more on that later)



Optimization 4: Microkernel

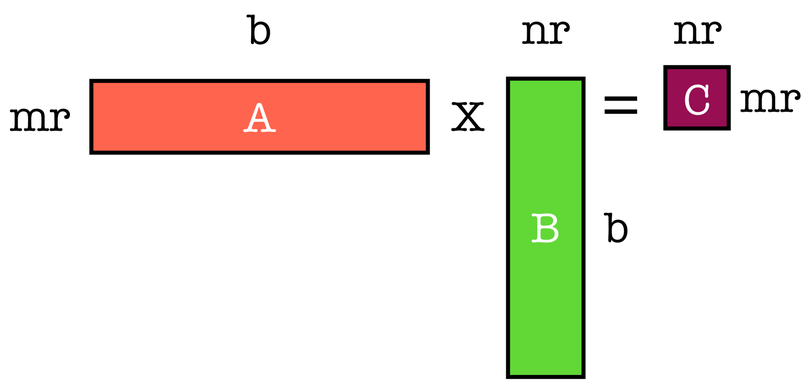
- **Microkernel**: highly-optimized GEMM kernel that computes a small fixed-size tile of the output
- Use repeated calls to this to implement multiplication of individual bxb blocks



Example microkernel

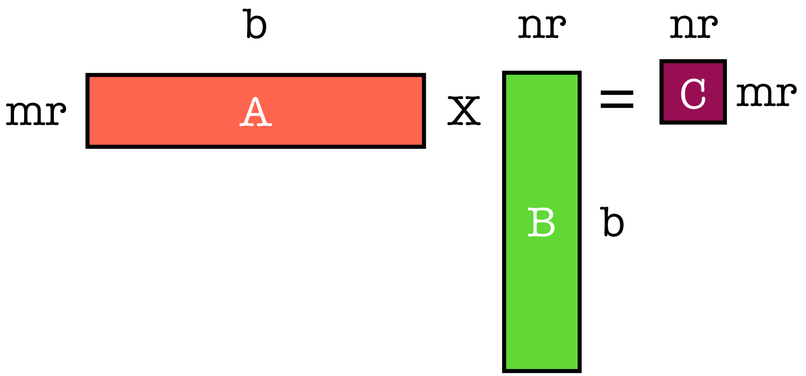
Optimization 4: Microkernel

- Job of the microkernel: **exploit data re-use at the register level**
- The microkernel loads mr x nr elements of C into registers and accumulates the results of A * B into the C registers
- **Key idea:** C persists in registers the whole time it's being accumulated into — only have to load from main memory into registers once
- Registers are really fast



Example microkernel

Optimization 4: Microkernel



- Sketch of what the microkernel **could** look like

Algorithm 1 Sketch of DGEMM Microkernel (4x8)

Require: Matrices A, B, C, all are $b \times b$

```
double C0, C1, C2, ..., C31                    ▷ Variables stored in registers
double Breg                                    ▷ Another variable stored in registers
C0 = C(0,0)                                    ▷ Load elements of C into registers
C1 = C(1,0)
...
C31 = C(3,7)
for k = 0 : b − 1 do                           ▷ Accumulate outer products into registers
    Breg = B(k,0)                              ▷ Load element of B into register
    C0 += A(0,k) * Breg                        ▷ Breg is re-used across the whole column of A
    C1 += A(1,k) * Breg
    ...
    Breg = B(k,1)                              ▷ Load element of B into register
    C4 += A(0,k) * Breg)
    C5 += A(1,k) * Breg)
    ...
    C31 += A(3,k) * Breg
end for
C(0,0) = C0                                    ▷ Write back to C
C(1,0) = C1
...
C(3,7) = C31
```

Only one load/store to registers — each iteration of loop accumulates into same registers

Load elements of C into registers

Each iteration computes outer product of a row of B and column of A
Every 4 multiply-adds, load a new element of B into the register

Write C back to memory

Optimization 4: Microkernel

- Another advantage of the microkernel: loops are explicitly **unrolled**
- Loop unrolling reduces overhead associated with checking loop bounds at the cost of increased binary size

Algorithm 1 Sketch of DGEMM Microkernel (4x8)

Require: Matrices A, B, C, all are $b \times b$
  double C0, C1, C2, ..., C31 ▷ Variables stored in registers
  double Breg ▷ Another variable stored in registers
  C0 = $C(0,0)$ ▷ Load elements of C into registers
  C1 = $C(1,0)$
  ...
  C31 = $C(3,7)$
  for $k = 0 : b-1$ **do** ▷ Accumulate outer products into registers
    Breg = $B(k,0)$ ▷ Load element of B into register
    C0 += $A(0,k) * Breg$ ▷ Breg is re-used across the whole column of A
    C1 += $A(1,k) * Breg$
    ...
    Breg = $B(k,1)$ ▷ Load element of B into register
    C4 += $A(0,k) * Breg)$
    C5 += $A(1,k) * Breg)$
    ...
    C31 += $A(3,k) * Breg$
  end for
  $C(0,0)$ = C0 ▷ Write back to C
  $C(1,0)$ = C1
  ...
  $C(3,7)$ = C31

Outer product loops are unrolled manually

Optimization 4: Microkernel

- Outer product approach provides ample opportunities for instruction level parallelism (ILP)
- Each Ci += A(…)B(…) can be computed independently — the corresponding instructions can be scheduled simultaneously

Algorithm 1 Sketch of DGEMM Microkernel (4x8)

Require: Matrices A, B, C, all are $b \times b$

```
double C0, C1, C2, ..., C31                    ▷ Variables stored in registers
double Breg                                    ▷ Another variable stored in registers
C0 = C(0,0)                                    ▷ Load elements of C into registers
C1 = C(1,0)
...
C31 = C(3,7)
for k = 0 : b − 1 do                           ▷ Accumulate outer products into registers
    Breg = B(k,0)                              ▷ Load element of B into register
    C0 += A(0,k) * Breg                        ▷ Breg is re-used across the whole column of A
    C1 += A(1,k) * Breg
    ...
    Breg = B(k,1)                              ▷ Load element of B into register
    C4 += A(0,k) * Breg)
    C5 += A(1,k) * Breg)
    ...
    C31 += A(3,k) * Breg
end for
C(0,0) = C0                                    ▷ Write back to C
C(1,0) = C1
...
C(3,7) = C31
```

All are independent instructions

Optimization 4: Microkernel

- It's also possible to vectorize the microkernel using SIMD intrinsics
- Sometimes the compiler will do this for you — double check the assembly
- Godbolt is a good tool for this

C source #1

A Save/Load + Add new... V Vim C

--NORMAL--

```c
/* Type your code here, or load an example. */

void daxpy(double * x, double * y, double a, int n) {
    for (int i=0; i<n; i++)
    {
        y[i] += x[i]*a;
    }
}
```

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1 -O3 -march=native

A Output... Filter... Libraries Overrides + Add new... Add tool...

```asm
daxpy:
        mov     rcx, rsi
        test    edx, edx
        jle     .L25
        cmp     edx, 1
        je      .L3
        lea     rsi, [rdi+8]
        mov     rax, rcx
        sub     rax, rsi
        cmp     rax, 16
        ja      .L27
.L3:
        movsx   rdx, edx
        xor     eax, eax
        sal     rdx, 3
.L9:
        vmovsd  xmm1, QWORD PTR [rdi+rax]
        vfmadd213sd     xmm1, xmm0, QWORD PTR [rcx+rax]
        vmovsd  QWORD PTR [rcx+rax], xmm1
        add     rax, 8
        cmp     rax, rdx
        jne     .L9
        ret
.L23:
        vzeroupper
```

Optimization 4: Microkernel

C source #1

A Save/Load + Add new... V Vim C C

--NORMAL--

```c
/* Type your code here, or load an example. */

void daxpy(double * x, double * y, double a, int n) {
    for (int i=0; i<n; i++)
    {
        y[i] += x[i]*a;
    }
}
```

x86-64 gcc 13.1 (Editor #1)

x86-64 gcc 13.1 -O3 -march=native

A Output... Filter... Libraries Overrides + Add new... Add tool...

```asm
daxpy:
        mov     rcx, rsi
        test    edx, edx
        jle     .L25
        cmp     edx, 1
        je      .L3
        lea     rsi, [rdi+8]
        mov     rax, rcx
        sub     rax, rsi
        cmp     rax, 16
        ja      .L27
.L3:
        movsx   rdx, edx
        xor     eax, eax
        sal     rdx, 3
.L9:
        vmovsd  xmm1, QWORD PTR [rdi+rax]
        vfmadd213sd     xmm1, xmm0, QWORD PTR [rcx+rax]
        vmovsd  QWORD PTR [rcx+rax], xmm1
        add     rax, 8
        cmp     rax, rdx
        jne     .L9
        ret
.L23:
        vzeroupper
```

The serial code on the left is automatically vectorized by the compiler

xmm1 etc are vector registers that can store 4 doubles — instructions with `v` are vector instructinos

Putting it All Together

Algorithm 2 Sketch of DGEMM Kernel

```
Require: Matrices A, B, C, all are n × n, block size b
  for i = 1 : n, step b do
      for j = 1 : n, step b do
          Pack block (i, j) of C into C_blk
          for k = 1 : n, step b do
              Pack block (i, k) of A into A_blk
              Pack block (k, j) of B into B_blk
              for ii = 1 : b, step mr do
                  for jj = 1 : b, step nr do
                      Call microkernel to update block (ii, jj) of C_blk
                  end for
              end for
          end for
      end for
  end for
```

Random Tips

- Start early — this can be hard if you've never done performance engineering before
- The optimizations listed here are not exhaustive and should not be taken as gospel — there are lots of things you can try — start with these and then explore
- Would highly recommend reading the Goto and Van de Geijn paper listed in the assignment description on the course website
- More detailed version of these slides: https://drive.google.com/file/d/14yhCdamrROEiXRd9fU74yVJENz3XfO3t/view
- Make a post on Ed if you are stuck!

Good Luck!